Trabajo práctico final

Comunicación inalámbrica - Uso de SPI y UART via DMA

Matías Morales Gariglio

18 de abril de 2026

1. Link repositorio

https://github.com/matutecno/FIUBA.git

2. Descripción General del Sistema

El sistema implementado consiste en un enlace de comunicación inalámbrica, el mismo será la base para el trabajo práctico final de la materia (gestión inteligente de recursos hídricos en entornos domésticos o industriales). Utilizando el transceptor **nRF24L01** en la banda ISM de 2.4 GHz, se establece una arquitectura de red inalámbrica para la adquisición y procesamiento de datos en tiempo real.

El proyecto se centra en la creación de un nodo de telemetría remoto (Slave) encargado de los sensores y un nodo central (Master) que procesa la información y gestiona las interfaces de salida. El sensor principal será el HC-SR04 (sensor de proximidad), cuya información será procesada y transmitida por el slave inalámbricamente. El master recibirá esta información y la mostrará de 3 formas distintas:

- Display luminoso led, tipo barras verticales
- UART en PC
- Alerta sonora cuando se llegue a un nivel mínimo

2.1. Arquitectura de archivos

Los archivos se acomodaron en la carpeta API, aquellos que corresponden exclusivamente tanto a maestro como a esclavo. Creé una carpeta denominada Personal_libs para portabilidad del código. Lo que hice en ports_master.h y ports_slave.h es cortar lo generado en stmCubeMx (defines de puertos y similares) del main.h y colocarlo allí. Lo malo de esto es que cualquier modificación futura en stmCubeMx generará nuevamente el main.h y podría generar un conflicto entre definiciones.

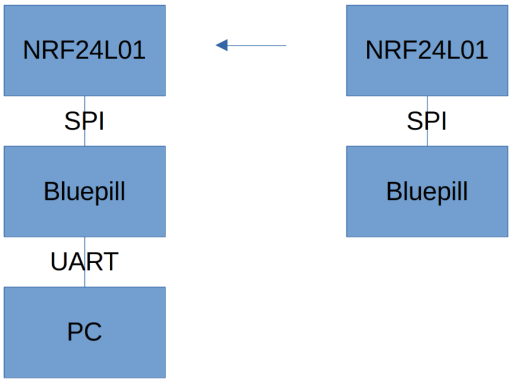

Figura 1: Diagrama en bloques del sistema propuesto.

```
---
MASTER (STM32F103C8T6)

Master_0/Master_0/
├── Core/
│   ├── Inc/
│   │   ├── main.h              ← generado CubeMX (pines + HAL)
│   │   ├── dma.h / gpio.h / spi.h / usart.h
│   │   └── stm32f1xx_it.h
│   └── Src/
│       ├── main.c
│       ├── dma.c / gpio.c / spi.c / usart.c
│       └── stm32f1xx_it.c
└── STM32CubeIDE/Drivers/API/
    ├── Inc/
    │   ├── ports_master.h      ← pines manuales + HAL (portabilidad)
    │   ├── API_display.h       ← display 74HC595 + estados
    │   └── API_parser.h        ← FSM recepción NRF + UART
    └── Src/
        ├── API_display.c
        └── API_parser.c
```

Figura 2: Archivos en Master

```
SLAVE_0 (STM32F103C8T6)

SLAVE_0/Slave_0/
├── Core/
│   ├── Inc/
│   │   ├── main.h              ← generado CubeMX
│   │   ├── dma.h / gpio.h / spi.h / usart.h / tim.h / adc.h
│   │   └── stm32f1xx_it.h
│   └── Src/
│       ├── main.c
│       ├── dma.c / gpio.c / spi.c / usart.c / tim.c / adc.c
│       └── stm32f1xx_it.c
└── STM32CubeIDE/Drivers/API/
    ├── Inc/
    │   ├── ports_slave.h       ← pines manuales + HAL (portabilidad)
    │   ├── API_parser.h        ← FSM medición JSN + transmisión NRF
    │   └── STM_init.h
    └── Src/
        └── API_parser.c
```

Figura 3: Archivos en slave

```
Personal_libs (compartidas MASTER y SLAVE)

Personal_libs/
├── NRF24L01/
│   ├── Inc/nrf24l01.h
│   └── Src/nrf24l01.c          ← driver SPI+DMA, TX/RX, bugs corregidos
├── JSNSR04T/
│   ├── Inc/jsnsr04t.h
│   └── Src/jsnsr04t.c          ← driver TIM2 Input Capture
├── Delay/
│   ├── Inc/u_delay.h
│   └── Src/u_delay.c           ← delay no bloqueante DWT
├── UART_MX/
│   ├── Inc/uart_mx.h
│   └── Src/uart_mx.c           ← wrapper UART CubeMX
└── UART/
    ├── Inc/API_uart.h
    └── Src/API_uart.c
```

Figura 4: Archivos en común, visibles entre maestro y esclavo

3. Periféricos utilizados

3.1. Escalabilidad y Visión de Futuro

El diseño del sistema se ha concebido bajo un paradigma de **arquitectura modular**, permitiendo su integración como un nodo periférico dentro de un ecosistema de domótica más robusto.

Al emplear el estándar de comunicaciones nRF24L01+, el sistema queda preparado para:

- **Topología de Red:** Evolucionar de un enlace punto a punto hacia una red tipo estrella o malla (Mesh), integrando múltiples nodos sensores distribuidos en una vivienda o planta industrial.
- **Interoperabilidad:** Actuar como un *gateway* inalámbrico que, mediante la interfaz UART de la unidad central, pueda conectarse a plataformas de control de nivel superior como *Home Assistant*, *OpenHAB* o servidores MQTT sobre una Raspberry Pi.
- **Modularidad de Hardware:** La separación de las etapas de potencia (control de cargas como el lavarropas) y las etapas de lógica asegura que el módulo pueda adaptarse a diferentes aplicaciones domésticas sin requerir un rediseño del núcleo de procesamiento.

4. Arquitectura del hardware

El sistema se divide en dos unidades funcionales con las siguientes características:
Nodo transmisor:

- **Unidad de Procesamiento:** Microcontrolador STM32F103C8T6 (Blue Pill).
- **Interfaz de Radio:** Módulo nRF24L01 conectado mediante bus SPI.
- **Sensores:** Entrada para sensor ultrasónico JSN-SR04T (impermeable) para medición de nivel/distancia.

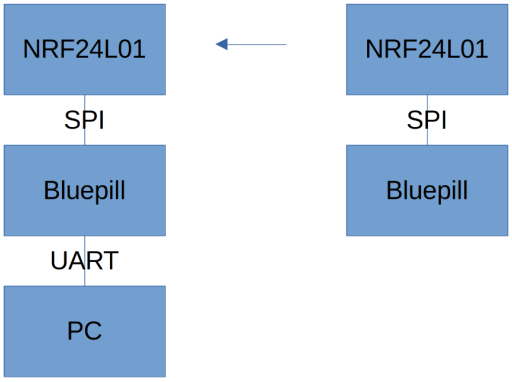

Figura 5: Diagrama en bloques del sistema propuesto.

Nodo receptor

- **Unidad de Procesamiento:** Microcontrolador STM32F103C8T6 (Blue Pill).
- **Interfaz de Radio:** Módulo nRF24L01 (SPI).
- **Interfaz de Usuario/PC:** Comunicación via UART hacia una PC para la visualización de datos mediante un terminal serie o interfaz gráfica.
- **Indicadores luminosos y sonoros:** Implementación de un display de segmentos verticales para indicar nivel y buzzer de alarma sonora.

5. Especificaciones de Software y Protocolos

Para la implementación de la lógica de control se utilizarán las siguientes herramientas y protocolos:

- **Capa de Transporte Inalámbrico:** Protocolo propietario de Nordic Semiconductor (Enhanced ShockBurst™), configurando:
  - Verificación de errores mediante CRC.
  - Reintento automático de paquetes (Auto-ACK).
- **Firmware:** Desarrollado en C utilizando las librerías HAL de STMicroelectronics.
- **SPI:** Configurado en Modo 0.
- **UART:** Configurada a 115200 bps para la transmisión de tramas hacia la PC.

6. Máquinas de estado

7. Objetivo del Trabajo Final

Este sistema servirá como plataforma base para la integración de un caudalímetro y el control de nivel de fluidos, permitiendo el monitoreo remoto de variables con una infraestructura de hardware propia y optimizada para entornos de baja potencia.

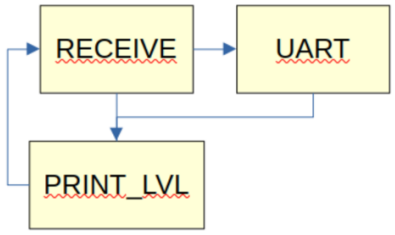

Figura 6: Máquina de estados del master.

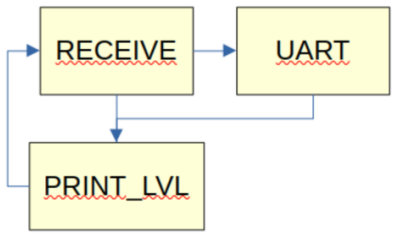

Figura 7: Máquina de estados del slave.